

FaCENA - UNNE

Facultad de Ciencias Exactas
y Naturales y Agrimensura
**UNIVERSIDAD NACIONAL
DEL NORDESTE**

TRABAJO PRÁCTICO

INTEGRADOR 2026:

SISTEMA DE SEMÁFOROS INTELIGENTES

MATERIA: Paradigmas y Lenguajes de Programación

GRUPO N. ° 22

INTEGRANTES:

- Canete Zacariaz Ana Paula
- Romero Ramírez Emanuel

INTRODUCCION

En el presente informe se describirá el desarrollo del Trabajo Práctico Integrador requerido para la regularización o promoción de la asignatura Paradigmas y Lenguajes de Programación. Este proyecto tiene como objetivo aplicar los conocimientos adquiridos durante el cursado e incorporar el aprendizaje de nuevas herramientas y tecnologías, siendo asignado a nuestro grupo el estudio del lenguaje OCaml.

¿En qué consiste la actividad? Consiste en estudiar y representar el funcionamiento de un sistema de semáforos inteligentes utilizando programación funcional. Para esto, se desarrolló una solución en Common Lisp que permite simular los diferentes estados del semáforo, los cambios entre colores y los tiempos de cada señal, además de obtener información sobre su funcionamiento para analizar los resultados. Luego de esto se realizó una investigación introductoria sobre el lenguaje OCaml, con el propósito de comparar sus características con las de Common Lisp y analizar diferentes enfoques para resolver un mismo problema utilizando herramientas propias de la programación funcional.

A lo largo de este informe no solo se explica cómo se realizó el trabajo y las funciones que se desarrollaron, sino que también se hace hincapié en los problemas que surgieron durante el desarrollo del programa y las conclusiones a las que se llegó a partir de esta experiencia.

DECISIONES DE DISEÑO ADOPTADAS

Para el desarrollo del sistema se optó por una solución basada en los principios de la programación funcional trabajados durante la cursada. Se buscó que cada función tuviera una responsabilidad específica y que pudiera resolver una tarea determinada sin depender del resultado de otras funciones.

La mayor parte de las funciones implementadas fueron diseñadas como funciones puras, es decir, funciones que para los mismos datos de entrada siempre producen el mismo resultado. Esta decisión permitió simplificar las pruebas realizadas durante el desarrollo y facilitar la detección de errores.

Asimismo, se evitó modificar datos existentes, priorizando la generación de nuevos resultados a partir de los valores recibidos como parámetros. De esta manera, se mantuvo un comportamiento predecible del sistema y una estructura más cercana al paradigma funcional.

Otra decisión importante fue dividir el problema en requerimientos independientes. En lugar de desarrollar una única función que resolviera todo el sistema, se implementaron distintas funciones encargadas de tareas específicas, como la gestión de transiciones, el cálculo de tiempos, el registro de auditoría y el análisis estadístico del ciclo semafórico. Esto permitió una mejor organización del código y una mayor facilidad para realizar pruebas y correcciones.

Por último, para representar los estados del semáforo se utilizaron símbolos y estructuras simples de datos, evitando complejidades innecesarias y priorizando la claridad del código y la comprensión del funcionamiento general del sistema.

DESARROLLO DE LA SOLUCIÓN EN COMMON LISP

Una vez definidas las decisiones de diseño y la forma en que se abordaría el problema se procedieron a implementar cada uno de los requerimientos planteados en la consigna mediante funciones independientes. Esta organización permitió que cada función cumpliera una tarea específica dentro del sistema, facilitando su desarrollo y su posterior verificación.

- **Requerimiento 1: Estado de transición**

El primer requerimiento consiste en controlar los cambios de estado del semáforo. Para ello se creó la función transición, que recibe el color actual y el color al que se desea pasar.

La función verifica si la transición solicitada es válida y devuelve la acción correspondiente. Por ejemplo, si el semáforo se encuentra en rojo y debe pasar a verde, se genera la acción de cambio adecuada. En caso de recibir una combinación no contemplada, la función devuelve una acción por defecto.

- **Requerimiento 2: Temporizador automático**

Para determinar qué color debe mostrar el semáforo en cada momento se implementó la función timer.

La misma recibe un valor de tiempo y calcula en qué parte del ciclo se encuentra el semáforo. Según el resultado obtenido, devuelve el color correspondiente. Para el desarrollo del trabajo se utilizaron los tiempos establecidos en la consigna: 90 segundos para rojo, 6 segundos para amarillo y 120 segundos para verde.

De esta manera es posible simular el funcionamiento automático del semáforo a lo largo del tiempo.

- **Requerimiento 3: Sistema de auditoría**

Con el fin de llevar un registro de los cambios realizados por el sistema se desarrolló la función registrar-auditoria.

Esta función muestra por pantalla información sobre cada transición realizada, indicando el instante en que ocurrió el cambio, el estado anterior y el nuevo estado.

- **Requerimiento 4: Análisis de ciclos semafóricos**

Para analizar el comportamiento general del semáforo se implementaron dos funciones.

- ~ La primera, duración-ciclo: calcula la duración total del ciclo semafórico sumando los tiempos asignados a cada color.
- ~ La segunda, recomendación-ciclo: utiliza ese resultado para evaluar si la duración del ciclo se encuentra dentro de valores razonables o si resulta demasiado corta o larga.

- **Requerimiento 5: Planificación temporal**

Este requerimiento busca calcular cuántos ciclos completos puede realizar el semáforo durante un determinado período de tiempo.

Para ello se desarrolló la función ciclos-por-tiempo, que recibe una cantidad de minutos, los convierte a segundos y calcula cuántos ciclos completos entran en ese intervalo.

- **Requerimiento 6: Informe de distribución temporal**

Por último, se implementó la función informe-distribución, cuya finalidad es calcular qué porcentaje del tiempo total del ciclo corresponde a cada color.

A partir de estos cálculos es posible observar cómo se distribuye el tiempo entre las señales roja, amarilla y verde, obteniendo una visión más clara del funcionamiento general del sistema.

FUNDAMENTOS DEL DISEÑO FUNCIONAL ADOPTADO

Además de cumplir con los requerimientos planteados en la consigna, durante el desarrollo del sistema se buscó aplicar los conceptos de programación funcional trabajados a lo largo de la materia. Por este motivo, antes de comenzar con la implementación fue necesario definir una forma de organización que permitiera resolver el problema de manera clara, ordenada y coherente con el paradigma utilizado.

La mayor parte de las funciones desarrolladas fueron implementadas como funciones puras, es decir, funciones que para los mismos datos de entrada producen siempre el mismo resultado, lo que se nos hizo útil para verificar funcionamiento de cada componente y analizar con mayor facilidad los resultados obtenidos durante el desarrollo.

Asimismo, se evitó modificar directamente los datos utilizados por el programa. En lugar de alterar valores existentes, las funciones generan nuevos resultados a partir de los parámetros recibidos y se organizan de manera que cada una cumpla una responsabilidad específica. Esta combinación permitió obtener un comportamiento más predecible y consistente, además de mantener una estructura ordenada que facilita tanto el mantenimiento como la comprensión del código

Si bien la mayoría de las funciones responden a estas características, existe una excepción representada por la función de auditoría. Su finalidad es registrar información por pantalla acerca de los cambios producidos durante la ejecución del sistema, por lo que genera un efecto externo y se considera una función impura dentro del conjunto de funciones desarrolladas.

PROCESO DE DEPURACIÓN Y CORRECCIÓN DE ERRORES

La implementación de los distintos requerimientos permitió obtener una primera versión funcional del sistema. Sin embargo, durante las pruebas fueron apareciendo errores tanto de sintaxis como de lógica que debieron ser analizados y corregidos para lograr el funcionamiento esperado.

Este proceso de revisión resultó fundamental no solo para detectar fallas en el programa, sino también para comprender mejor el comportamiento de Common Lisp y las herramientas utilizadas. A continuación, se describen algunos de los inconvenientes encontrados durante el desarrollo, las causas que los originaron y como lo solucionamos:

- Error 1: Intentamos desarrollar el informe de distribución temporal para conocer el impacto porcentual de cada color en una hora de funcionamiento del semáforo.

Código inicial:

```
(defun informe-distribucion-impuro ()  
  (let ((total-ciclo 216.0))  
    (format t "Porcentaje Rojo: ~a%~%" (* (/ 90 total-ciclo) 100))  
    (format t "Porcentaje Amarillo: ~a%~%" (* (/ 6 total-ciclo) 100))  
    (format t "Porcentaje Verde: ~a%~%" (* (/ 120 total-ciclo) 100))))
```

En esta primera versión, la función utilizaba `format t` para mostrar los porcentajes directamente en la terminal. El inconveniente de esto era que el valor de retorno de la función era `NIL` en lugar de los datos numéricos calculados. Para que la función permitiera reutilizar sus resultados en otras partes del programa, se requería que devolviera una estructura de datos en lugar de limitarse a imprimir texto en la consola.

Código (Solución final):

```
(defun informe-distribucion ()  
  (let ((total-ciclo 216.0))
```

```
(list  
(list 'porcentaje-rojo (* (/ 90 total-ciclo) 100))  
(list 'porcentaje-amarillo (* (/ 6 total-ciclo) 100))  
(list 'porcentaje-verde (* (/ 120 total-ciclo) 100))))
```

- Error 2: Intentamos determinar el color del semáforo correspondiente a un instante específico en tiempo Unix (timestamp), evaluando el transcurso de los segundos dentro del ciclo total de 216 segundos.

Código inicial:

```
(defun timer-imperativo (timestamp)  
  (let ((tiempo-restante timestamp))  
    (loop while (>= tiempo-restante 216)  
      do (setq tiempo-restante (- tiempo-restante 216)))  
    (cond  
      ((< tiempo-restante 90) 'rojo)  
      ((< tiempo-restante 96) 'amarillo)  
      (t 'verde))))
```

Este diseño inicial utilizaba un bucle iterativo que modificaba una variable local mediante sets para restar sucesivamente la duración del ciclo hasta obtener el residuo. Al contrastar esto con las restricciones del trabajo práctico, se identificó que el uso de operadores como setq no estaba permitido, lo que obligó a buscar una solución matemática que no requiriera alterar el valor de las variables locales.

Código (Solución final):

```
(defun timer (timestamp)  
  (let ((posicion-en-ciclo (mod timestamp 216)))  
    (cond  
      ((< posicion-en-ciclo 90) 'rojo)  
      ((< posicion-en-ciclo 96) 'amarillo)
```



```
(t 'verde))))
```

- **Error 3:** Se intención validar si la duración total de un ciclo de semáforo se encontraba dentro del rango óptimo psicológico (entre 35 y 150 segundos) para mitigar el estrés vehicular.

Código inicial (Diseño erróneo):

```
(defun recomendacion-ciclo-acoplado (duracion)
  (cond
    ((< 216 35) "Evitar: El ciclo es demasiado corto.")
    ((> 216 150) "Evitar: El ciclo es demasiado largo.")
    (t "Correcto: El ciclo está dentro del rango óptimo.)))
```

El problema fue escribir el número fijo 216 directamente dentro de las condiciones de evaluación de la estructura cond. Al hacer esto, la función ignoraba por completo el parámetro duracion que recibía como argumento, devolviendo siempre el mismo resultado predeterminado e impidiendo su uso para validar cualquier otro ciclo de tiempo diferente.

Código (Solución final):

```
(defun recomendacion-ciclo (duracion)
  (cond
    ((< duracion 35) "Evitar: El ciclo es demasiado corto (menor a 35s).")
    ((> duracion 150) "Evitar: El ciclo es demasiado largo (mayor a 150s).")
    (t "Correcto: El ciclo está dentro del rango óptimo psicológico.)))
```

- **Error 4:** Intentamos calcular la cantidad de ciclos semafóricos completos que ocurren dentro de un intervalo de tiempo determinado expresado en minutos.

Código inicial:

```
(defun ciclos-por-tiempo-incorrecto (minutos)
  (let ((segundos-totales (* minutos 60)))
    (round (/ segundos-totales 216))))
```

Al utilizar la función round, el entorno aplicaba un redondeo al entero más cercano. Esto causaba un desfase en la lógica del

sistema: si el resultado de la división era, por ejemplo, 2.6 ciclos, la función devolvía 3. Esto representaba un error de estimación, ya que el tercer ciclo no se había completado al 100%. Para registrar únicamente los ciclos finalizados en su totalidad, se requería una aproximación matemática por piso.

Código(Solución final):

```
(defun ciclos-por-tiempo (minutos)
  (let ((segundos-totales (* minutos 60)))
    (floor (/ segundos-totales 216))))
```

- Error 5: Intentamos calcular la duración total de un ciclo a partir de sus tres fases temporales y emitir una recomendación técnica sobre su viabilidad en el tránsito urbano.

Código inicial :

```
(defun evaluar-semaforo-monolitico (t-rojo t-amarillo t-verde)
  (let ((total (+ t-rojo t-amarillo t-verde)))
    (cond
      ((< total 35) "Evitar: El ciclo es demasiado corto.")
      ((> total 150) "Evitar: El ciclo es demasiado largo.")
      (t "Correcto: El ciclo está dentro del rango óptimo."))))
```

En este diseño inicial, la suma aritmética y la validación del rango óptimo se encontraban acopladas en una sola estructura. El inconveniente de este metodo era la falta de reutilización. El valor de la duración total quedaba confinado al ámbito interno de esta función, impidiendo que otros módulos del programa pudiesen consultar la duración total de forma aislada. La solución requería segregar el cálculo de la evaluación lógica.

Código (Solución final):

```
(defun duracion-ciclo (tiempo-rojo tiempo-amarillo tiempo-verde)
  (+ tiempo-rojo tiempo-amarillo tiempo-verde))

(defun recomendacion-ciclo (duracion)
  (cond
```

((< duracion 35) "Evitar: El ciclo es demasiado corto (menor a 35s).")

((> duracion 150) "Evitar: El ciclo es demasiado largo (mayor a 150s).")

(t "Correcto: El ciclo está dentro del rango óptimo psicológico.")))

INVESTIGACIÓN SOBRE EL LENGUAJE OCAML

Como parte de la tercera fase del trabajo, se realizó una investigación introductoria sobre el lenguaje OCaml, con el objetivo de comprender sus principales características y compararlas con las de Common Lisp.

OCaml es un lenguaje de programación que investigamos como parte de este trabajo. Pertenece a la familia de lenguajes ML (Meta Language), un conjunto de lenguajes funcionales desarrollados originalmente para la demostración de teoremas y caracterizados por su sistema de tipos estáticos con inferencia automática, y está orientado principalmente a la programación funcional, aunque también permite utilizar elementos de programación imperativa y orientada a objetos. Durante la investigación nos llamó la atención su sistema de tipos con inferencia automática, ya que permite que el compilador detecte muchos errores sin que el programador tenga que indicar explícitamente el tipo de cada variable.

Gracias a estas características, OCaml suele utilizarse en proyectos donde la seguridad, la confiabilidad y el rendimiento son aspectos importantes. Sus aplicaciones pueden encontrarse en áreas como el desarrollo de compiladores, herramientas de análisis de código, sistemas financieros, ciberseguridad, verificación formal de software e investigación académica.

Entre las organizaciones y empresas que utilizan o han utilizado OCaml en distintos proyectos pueden mencionarse Jane Street, reconocida firma del sector financiero especializada en trading electrónico; LexiFi, dedicada al desarrollo de software para productos financieros; Ahrefs, empresa vinculada al análisis de

posicionamiento web; y Tarides, organización enfocada en el desarrollo y mantenimiento del ecosistema OCaml.

Durante este trabajo también se realizó una versión en OCaml de las funciones de transición de estados y del temporizador que habían sido desarrolladas originalmente en Common Lisp. Esto nos permitió tener un primer contacto con el lenguaje y comparar cómo se resuelve un mismo problema utilizando herramientas y enfoques diferentes en cada caso.

RESPUESTAS A LOS EJES COMPARATIVOS DE LA FASE 3

- 1. Al igual que Haskell, OCaml usa tipos estáticos, pero cuenta con Inferencia de Tipos. ¿Tuvieron que declarar explícitamente de qué tipo eran las funciones o el compilador lo dedujo solo? Muestran cómo lo interpreta el entorno.*

Durante la implementación de las funciones en OCaml no fue necesario declarar explícitamente los tipos de datos utilizados. El compilador pudo deducir automáticamente el tipo de los parámetros y del valor devuelto a partir de las operaciones realizadas dentro de cada función.

Por ejemplo, al definir la función *timer*, OCaml interpretó automáticamente que recibe un valor entero y devuelve una cadena de texto:

val timer : int -> string = <fun>

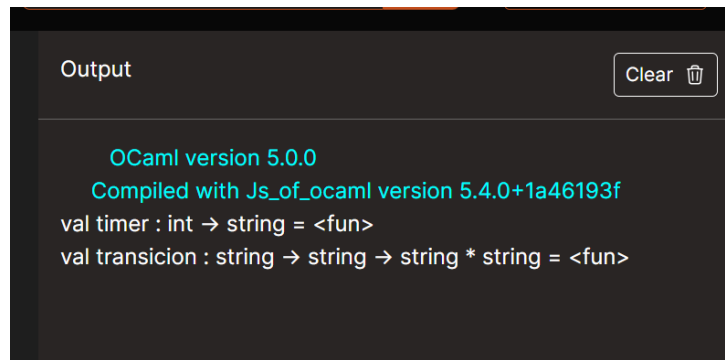
- ~ **timer es una función.**
- ~ **Recibe un int (entero).**
- ~ **Devuelve un string (cadena de texto).**

De manera similar, para la función *transición*, el entorno identificó que recibe dos cadenas de texto y devuelve una tupla compuesta por dos cadenas:

val transicion : string -> string -> string * string = <fun>

La inferencia de tipos es una de las características más importantes de OCaml, ya que permite trabajar con tipado estático sin necesidad de declarar manualmente los tipos en la mayoría de los casos.

Para comprobar cómo funcionaba la inferencia de tipos en OCaml, se probaron las funciones desarrolladas. Al ejecutarlas, el propio entorno indicó automáticamente qué tipos de datos recibía y devolvía cada función, sin que fuera necesario especificarlos al momento de programarlas. En la siguiente imagen puede observarse el resultado obtenido.

A screenshot of a terminal window with a dark background. The title bar says "Output". In the top right corner, there is a "Clear" button with a trash icon. The output text is as follows:

```
OCaml version 5.0.0
Compiled with Js_of_ocaml version 5.4.0+1a46193f
val timer : int → string = <fun>
val transicion : string → string → string * string = <fun>
```

2. OCaml es un lenguaje orientado a expresiones. ¿Cómo cambia esto la estructura de control de flujos (como el uso de match...with) comparado con los condicionales de Lisp?

Durante el desarrollo observamos que una de las principales diferencias entre ambos lenguajes se encuentra en la forma de expresar las decisiones dentro del programa.

En Common Lisp utilizamos la estructura cond, que evalúa distintas condiciones hasta encontrar una que resulte verdadera. Por ejemplo, en la función transicion se verifican diferentes combinaciones de estados mediante expresiones lógicas.

En OCaml, en cambio, utilizamos la estructura match ... with, que permite comparar directamente los valores recibidos con distintos casos posibles.

```
match (color_actual, cambiar_a) with
| ("en-rojo", "verde") -> ...
```

| ("en-verde", "amarillo") -> ...

| ("en-amarillo", "rojo") -> ...

| _ -> ...

Al realizar la adaptación de la función transicion, notamos que este mecanismo permite visualizar todos los casos posibles de manera más ordenada. Mientras que en Lisp se construyen condiciones que deben evaluarse una por una, en OCaml se describen directamente las situaciones que pueden ocurrir y la acción correspondiente para cada una.

Por lo tanto, aunque ambos enfoques permiten resolver el mismo problema, el uso de match ... with resultó más claro para representar las distintas transiciones del semáforo dentro de nuestro trabajo.

CONCLUSIÓN

A lo largo de este Trabajo Práctico Integrador pudimos poner en práctica los conceptos de programación funcional que vimos durante la cursada. Mientras íbamos resolviendo cada uno de los requerimientos, entendimos mejor la importancia de crear funciones de una forma clara, reutilizables y fáciles de mantener, siguiendo la lógica del paradigma funcional.

También aprendimos mucho durante el proceso de prueba y corrección del programa. Nos encontramos con distintos errores y desafíos para analizar y resolver, lo que nos ayudó a comprender mejor cómo funciona el lenguaje y responder dudas que se nos cruzaban.

Además, la investigación sobre OCaml nos permitió conocer otra herramienta de programación funcional y comparar sus características con las de Common Lisp. Esto nos ayudó a tener una visión más amplia de diferentes maneras de abordar y resolver un mismo problema.

En general, consideramos que este trabajo fue una experiencia muy útil porque nos permitió aplicar los conocimientos teóricos en un caso práctico, reforzar conceptos vistos en clase y desarrollar habilidades que seguramente nos servirán en futuras materias y proyectos relacionados con la programación.

Nota de Alumna Cañete Zacariaz Ana Paula: Profesor/a mi video se grabo sin audio por un problema que tuve con la computadora y no me di cuenta hasta que lo estaba terminando de editar. Lo subo para poder mandarle el link, pero lo voy a rehacer <https://youtu.be/CReGig7TV4U>